Struktur Pemrograman Python (Bagian 2)

Memahami nilai, tipe data, operator, dan struktur fundamental dalam pemrograman Python.

Nilai dan Tipe Data

Apa itu Nilai?

Nilai adalah hal paling mendasar seperti huruf atau angka yang akan dimanipulasi oleh program. Contoh: angka 2 (hasil dari 1 + 1) dan "Hello Python!".

Nilai-nilai tersebut memiliki tipe data berbeda: 2 adalah **integer**, sedangkan "Hello Python!" adalah **string** karena terdiri dari kata-kata dalam tanda kutip dua.

Mengidentifikasi Tipe

Interpreter dapat memberitahu tipe data menggunakan fungsi built-in `type()`:

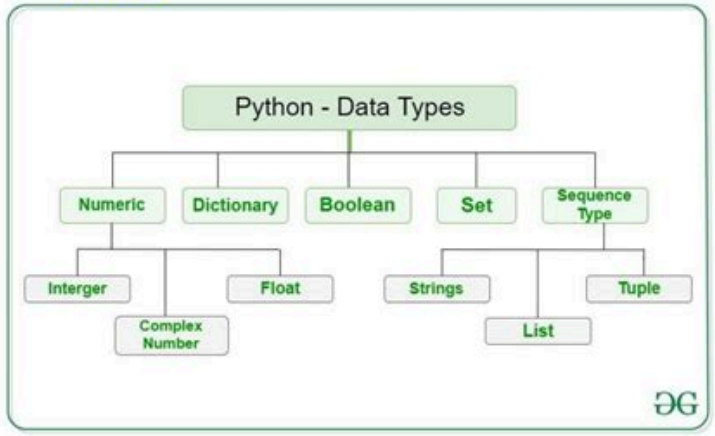
```
>>> type("Hello Python!")
<'type string'>
>>> type(5)
<'type int'>
>>> a="Belajar Python"
>>> type(a)
<'type string'>
```

Python Data Types

Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data.

Following are the standard or built-in data type of Python:

- Numeric
- Sequence Type
- Boolean
- Set
- Dictionary



Numeric

In Python, numeric data type represent the data which has numeric value. Numeric value can be integer, floating number or even complex numbers. These values are defined as `int`, `float` and `complex` class in Python.

- **Integers** – This value is represented by int class. It contains positive or negative whole numbers (without fraction or decimal).
- **Float** – This value is represented by float class. It is a real number with floating point representation. It is specified by a decimal point.
- **Complex Numbers** – Complex number is represented by complex class. It is specified as *(real part) + (imaginary part)j*. For example – 2+3j

Example:
Note – `type()` function is used to determine the type of data type.

Tipe Data Number

Tipe data Number merepresentasikan nilai-nilai berupa angka. Python menggolongkan Integer (bilangan bulat) dan Floating-point (bilangan desimal) ke dalam tipe data Number.

Integer

Bilangan bulat tanpa desimal

```
>>> 123 + 789
912
>>> 6 * 34
204
>>> 3 ** 2
9
```

Float

Bilangan dengan titik desimal

```
>>> phi=3.14
>>> type(phi)
<'type float'>
>>> 3.245 * 3
9.735
```

Pembulatan

Perhitungan menghasilkan nilai 0.0-0.9 dibulatkan ke bawah

```
>>> 9 / 2
4
```

Operator Penugasan

Operator penugasan (=) digunakan untuk memasukkan nilai ke dalam variabel. Tidak ada hasil yang muncul sampai statement selanjutnya dijalankan.

Penugasan Tunggal

```
>>> a = 8
>>> b = 3.5
>>> a * b
28.0
```

Penugasan Simultan

```
>>> x = y = z = 20
>>> x
20
>>> y
20
>>> z
20
```

Tipe Data String

Python mampu melakukan manipulasi string yang dapat diekspresikan dengan berbagai cara. Penulisan nilai string menggunakan tanda petik satu (') atau tanda petik dua (").

String Sederhana

```
>>> "Selamat datang"
'Selamat datang'
>>> 'Selamat datang'
'Selamat datang'
```

Multiple Line dengan \n

```
>>> text = "ini adalah contoh \n\
... penggunaan multiple line\n\
... di python"
>>> print text
ini adalah contoh
penggunaan multiple line
di python
```

Triple Quotes

```
>>> print """
... my name is python
... i'm an object oriented
... programming language
... """
```

Python String Methods

```
"Hello World".capitalize()  -->  Hello world
"Hello World".casefold()  -->  hello world
"Hello World".count("o")  -->  2
"Hello World".find("World") -->  6
"Hello World".index("Hello")  -->  0
"Hello World".isalnum()  -->  False
"Hello World".isalpha()  -->  False
"Hello World".isascii()  -->  True
"Hello World".isdecimal()  -->  False
"Hello World".isdigit()  -->  False
"Hello World".isidentifier()  -->  False
"hello world".islower()  -->  True
"Hello World".isnumeric()  -->  False
"Hello World".isprintable()  -->  True
```

Operasi pada String

Pada umumnya tidak dapat melakukan operasi matematika pada string, walaupun string tersebut berupa angka. Namun, operator tertentu memiliki fungsi khusus untuk string.

01

Operator Tambah (+)

Menggabungkan dua string atau lebih

```
>>> hadir = "Peserta sebanyak 1"
>>> banyak = "100"
>>> print "hadir" + hadir + banyak
hadir Peserta sebanyak 1100
>>> 'universitas' + 'gunadarma'
'universitasgunadarma'
```

02

Operator Kali (*)

Menggandakan string dengan integer

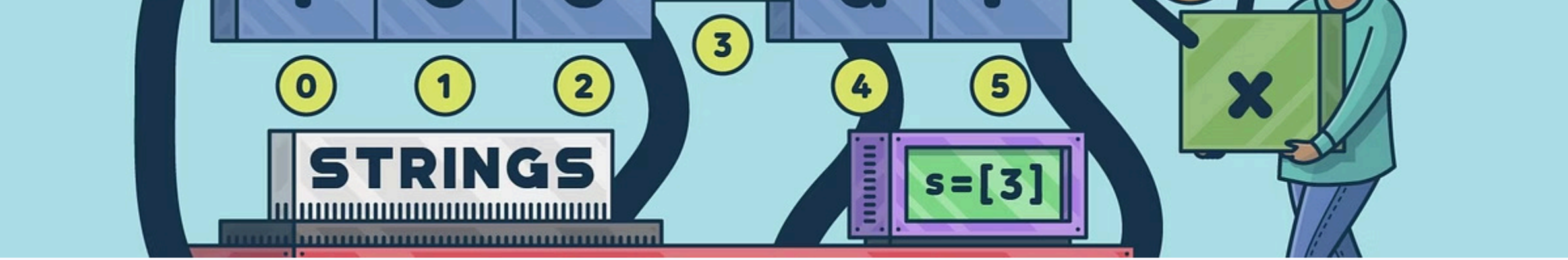
```
>>> "ulang" * 3
'ulangulangulang'
```

03

Tanda Koma (,)

Sebagai pemisah (spasi) antar string

```
>>> print "ulang", 3, 4, 5
ulang 3 4 5
```



Indexing String

String dalam Python dianggap sebagai array of character. Karakter pertama berindex 0, karakter kedua berindex 1, dan seterusnya.

```
>>> kata = "Gunadarma"
>>> kata[0]
'G'
>>> kata[5]
'a'
>>> kata[0:5]
'Gunad'
>>> kata[4:8]
'darm'
>>> kata[:7]
'Gunadar'
>>> kata[5:]
'arma'
```

Catatan Penting

Elemen karakter dalam string **tidak dapat digantikan**, tetapi dapat ditambahkan dengan operator (+).

```
>>> 'pert' + kata[5:]
'pertarma'
```

Tipe Data List

List adalah jenis data campuran yang bisa memiliki komponen penyusun berbeda-beda. List dibuat dengan tanda kurung siku [ ].

Dapat Diganti

Komponen penyusunnya bisa diganti

Dapat Dibaca

Komponen dapat dibaca dan dimanipulasi langsung

Dapat Ditambah

Komponen penyusunnya bisa ditambah

Nested List

Komponen dapat berupa list lain

```
>>> a = ['lab', 'TI', 2010, 'J1']
>>> a
['lab', 'TI', 2010, 'J1']
>>> a[3]
'J1'
>>> len(a)
4
```

Operasi List

Python dapat mengelompokkan beberapa tipe data berbeda menjadi satu kelompok. List dianalogikan sebagai array dengan urutan pengaksesan dimulai dari 0.

Pengaksesan List

```
>>> a = ["satu", 2, 3.0, "empat"]
>>> print a
['satu', 2, 3.0, 'empat']
>>> a[0]
'satu'
>>> a[1]
2
>>> a[-2]
3.0
>>> a[3]
'empat'
```

Slicing List

```
>>> a[0:2]
['satu', 2]
>>> a[-4:-1]
['satu', 2, 3.0]
```

Tanda titik dua ":" berarti dimulai dari indeks sampai indeks ke-n (batas indeks-n tidak ditampilkan).

Manipulasi List

1

Penambahan

```
>>> a + ['lima', 'enam']
['satu', 2, 3.0, 'empat',
'lima', 'enam']
```

2

Penggandaan

```
>>> 3*a[:3] + ['tujuh']
['satu', 2, 3.0, 'satu', 2,
3.0, 'satu', 2, 3.0, 'tujuh']
```

3

Perubahan

```
>>> a[2] = a[1] + 5
>>> a
['satu', 2, 7, 'empat']
```

Untuk menggantikan sebagian anggota list secara berurutan juga diperbolehkan. Gunakan fungsi `len()` untuk mengetahui jumlah elemen anggota List.

Metode-Metode List

1

append(x)

Menambahkan satu elemen di akhir list

2

extend(L)

Menggantikan seluruh anggota dengan elemen list L

3

insert(i, x)

Menyisipkan elemen pada posisi tertentu

4

remove(x)

Menghilangkan satu anggota list

5

pop([i])

Menghilangkan anggota pada posisi tertentu

6

index(x)

Mengembalikan nilai indeks suatu anggota

7

count(x)

Memeriksa jumlah x dalam list

8

sort()

Mengurutkan anggota list

9

reverse()

Membalik urutan list

Latihan: Program Kalender Bulan

Buatlah program kalender bulan dengan tampilan yang meminta input bulan (1-12) dan menampilkan nama bulan yang dipilih.

Contoh Output

```
Bulan apa [1-12]? 2
Bulan yang Anda pilih ? Februari
```

Petunjuk: Menggunakan List sebagai daftar nama bulan.

Solusi

```
Bulan = ['Januari', 'Februari',
'Maret', 'April', 'Mei', 'Juni',
'Juli', 'Agustus', 'September',
'Oktober', 'Nopember', 'Desember']

Pilih = input("Bulan apa [1-12]? ")

if 1 <= Pilih <= 12 :
 print "Bulan yang Anda pilih ?",
 Bulan[Pilih-1]
```

Tipe Data Dictionary

Berbeda dengan list yang memakai indeks angka, dictionary memakai key untuk merujuk pada isi variabelnya. Dictionary dideklarasikan dengan tanda kurung kurawal { }.

Contoh Dictionary

```
>>> D = {'food': 'spam',
'quality': 4, 'color': 'blue'}
>>> D
{'food': 'spam', 'quality': 4,
'color': 'blue'}
>>> D['color']
'blue'
```

Membuat Dictionary Kosong

```
>>> status = {}
>>> status['nama']='python'
>>> status['desc']='programming
language'
>>> status['age']=6
>>> status
{'nama': 'python',
'desc': 'programming language',
'age': 6}
```

Operator dan Operand

Operator adalah simbol-simbol khusus yang merepresentasikan komputasi seperti penambahan dan perkalian. Nilai yang digunakan oleh operator disebut **operand**.

Ekspresi Valid

Berikut adalah ekspresi-ekspresi yang benar dalam Python:

- 20+3
- hour-1
- hour*60+minute
- minute/60
- 5**2
- (5+9)*(15-7)

Pembagian Integer

```
>>> minute = 66
>>> minute / 60
1
```

Hasil adalah 1 bukan 1.1 karena Python melakukan pembagian integer.

Pembagian Float

```
>>> minute = 66.0
>>> minute / 60
1.1000000000000001
```

Gunakan float untuk hasil desimal yang akurat.

Operator Logika

Terdapat 3 operator logika: and, or, dan not. Arti ketiga operator ini sama dengan arti sebenarnya dalam bahasa Inggris.

Operator AND

`x > 8 and x < 20` adalah benar jika kedua kondisi terpenuhi, yaitu x lebih besar dari 8 DAN lebih kecil dari 20.

Operator OR

`x % 2 == 0 or x % 3 == 0` menghasilkan nilai true jika salah satu kondisi benar, yaitu x dapat dibagi 2 ATAU 3.

Operator NOT

`not (x > y)` me-negasikan ekspresi boolean. Bernilai true jika `(x > y)` bernilai false.

Catatan: Semua angka bukan nol (0) diinterpretasikan sebagai true. Contoh: `x = 5; x and 1` menghasilkan 1, sedangkan `y = 0; y and 1` menghasilkan 0.

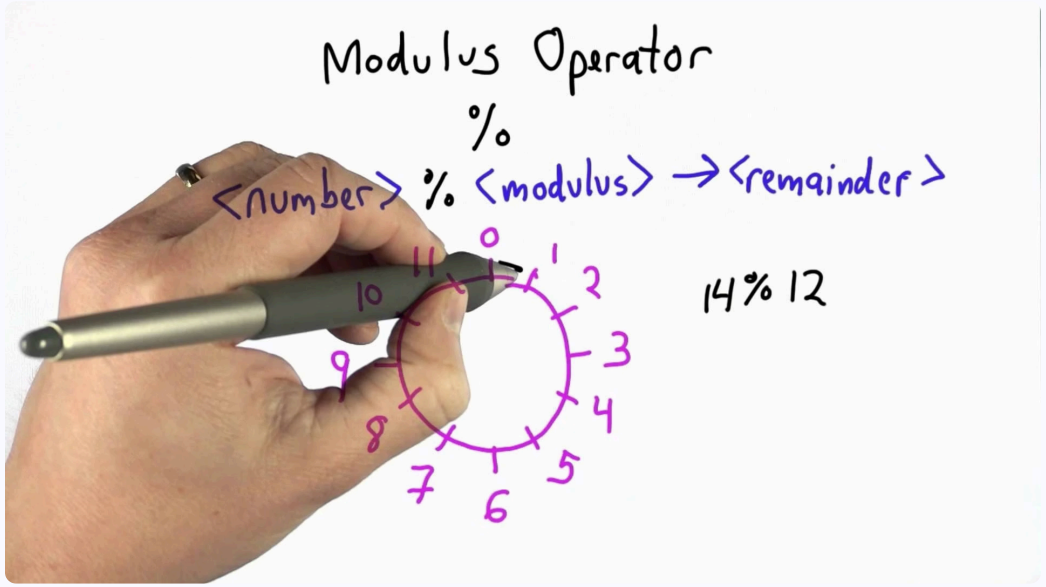
Operator Modulus

Operator modulus bekerja pada bilangan integer dan menghasilkan nilai sisa hasil pembagian operan pertama dengan operan kedua. Di Python, operator modulus diwakili simbol persentase (%).

Contoh Penggunaan

```
>>> pembagian = 5 / 3
>>> print(pembagian)
1
>>> sisa = 5 % 3
>>> print(sisa)
2
```

Jadi 5 dibagi 3 hasilnya adalah **1 sisa 2**.



ORDER OF OPERATIONS

The order of operations tells you the sequence to follow when you are performing operations in a mathematical expression.

Aturan Presedansi Operasi

Jika terdapat lebih dari satu operator dalam ekspresi, aturan operasi tergantung dari aturan presedansi. Python mengikuti aturan presedansi matematika pada umumnya.

Kurung (Tertinggi)

Operasi dalam kurung diproses terlebih dahulu. Contoh: `4 * (5+4) = 36`

Pangkat

Tanda pangkat (**) memiliki presedansi tinggi. Contoh: `3**2 + 8 = 17` bukan 13

Perkalian & Pembagian

Presedansi sama, didahulukan dari penambahan/pengurangan. Contoh: `1 + 3 * 2 = 7`

Penambahan & Pengurangan

Presedansi sama, dievaluasi dari kiri ke kanan. Contoh: `4 + 5 - 2 = 7`

Tabel Operator Aritmatika

Operator	Deskripsi	Contoh	Hasil
`*`	Perkalian	7 * 3	21
`/`	Pembagian	7 / 3	2
`%`	Modulus	7 % 3	1
`+`	Penjumlahan	7 + 3	10
`-`	Pengurangan	7 - 3	4

Operator Perbandingan & Penugasan

Operator Perbandingan

Operator	Deskripsi	Contoh	Hasil
`>=`	Lebih besar atau sama dengan	7 >= 9	FALSE
`<=`	Lebih kecil atau sama dengan	3 <= 8	TRUE
`!=`	Tidak sama dengan	1 != 10	TRUE
`<`	Lebih kecil	14 < 6	FALSE
`>`	Lebih besar	5 > 3	TRUE
`==`	Sama dengan	4 == 4	TRUE

Operator Penugasan

Operator	Contoh	Sama dengan
`*=`	x *= 100	x = x * 100
`/=`	x /= 100	x = x / 100
`%=`	x %= 100	x = x % 100
`+=`	x += 100	x = x + 100
`-=`	x -= 100	x = x - 100

Ringkasan Struktur Pemrograman Python

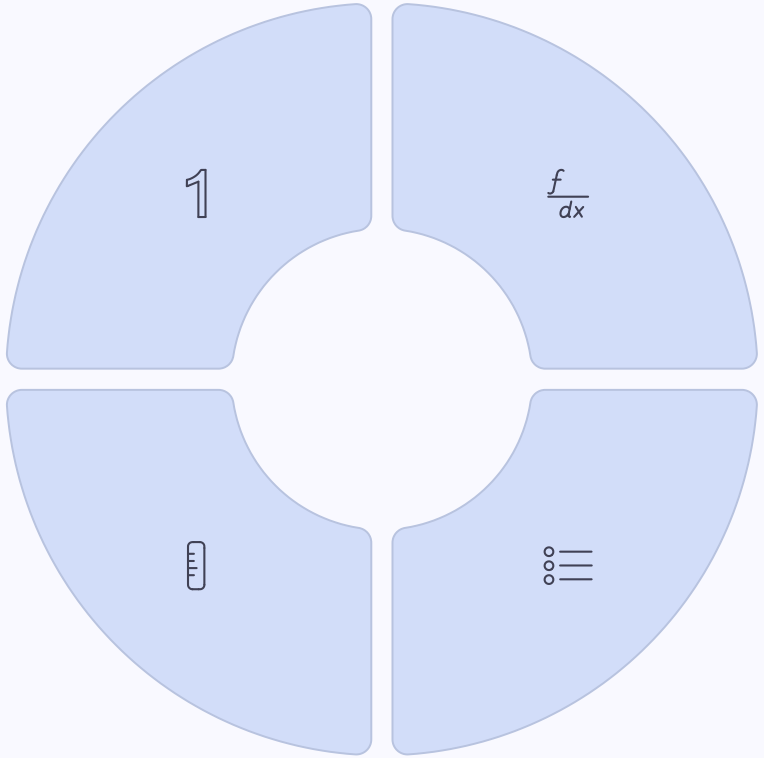
Anda telah mempelajari konsep-konsep fundamental dalam pemrograman Python yang mencakup nilai, tipe data, dan operator.

Tipe Data

Number, String, List, dan Dictionary sebagai fondasi manipulasi data

Operator

Aritmatika, logika, perbandingan, dan penugasan untuk komputasi



Aturan Presedansi

Urutan evaluasi operasi untuk hasil yang akurat

Struktur Data

List dan Dictionary untuk penyimpanan data kompleks

Dengan memahami konsep-konsep ini, Anda siap untuk melanjutkan ke topik pemrograman Python yang lebih advanced.